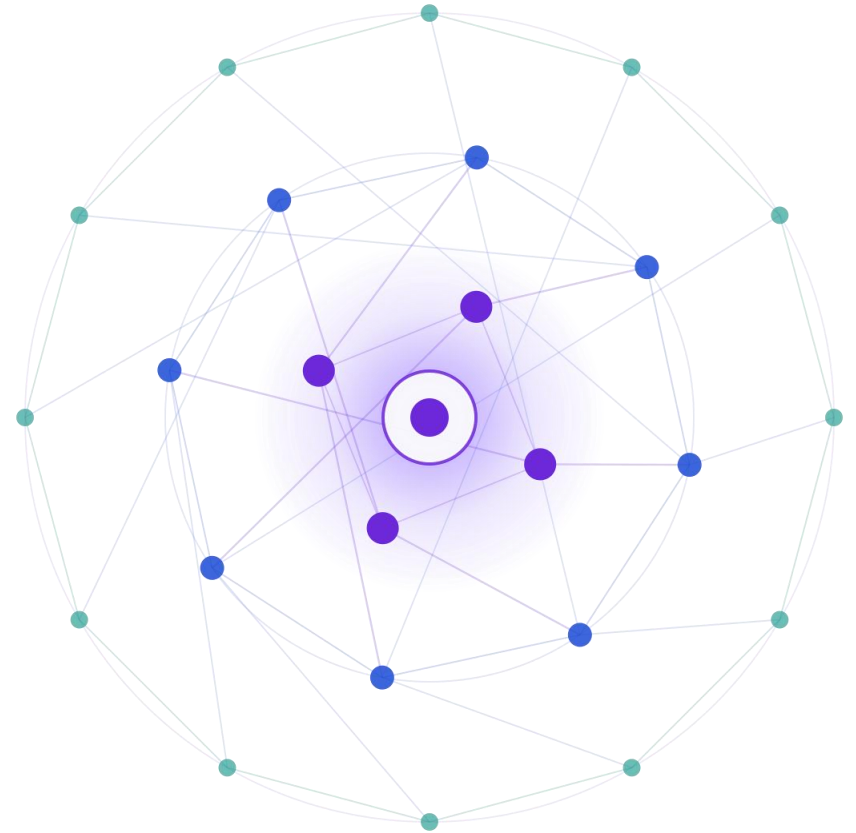


Generative AI Hackathon

3rd Edition 2026 – 27

- **Theme ID** - Theme 01 (Agentic Code Intelligence)
- **Team Name** - CodeNexus
- **College Name** - Vellore Institute of Technology
- **Member Name & Email 1**- Mithun Venkatesan
(mithun.venkatesan2023@vitstudent.ac.in)
- **Member Name & Email 2**- Kavın M (kavin.m2023@vitstudent.ac.in)
- **Member Name & Email 3**- Akhiil Dheep B
(akhiildheep.b2023@vitstudent.ac.in)
- **Member Name & Email 4**- None
- **Submission Github link** - https://github.com/Mithun-Newt/agentic_code_intelligence.git



Theme

Domain & Problem Context: Theme 01 — Agentic Code Intelligence (Code Search & Retrieval). In agentic software development, autonomous coding agents spend significant latency and token budget identifying relevant repository files and code blocks before modifying code.

- **The Core Retrieval Bottleneck:** Overcoming the severe semantic-to-syntactic divide: mapping high-level natural language requirements, bug reports, and edge-case descriptions to terse algorithmic code snippets across thousands of candidates.
- **Official Samsung Hackathon Scope (Theme 01):**
 - **In-Scope (P0 Shipped):** Dense semantic retrieval, asymmetric task instruction prompt conditioning, empirical model selection, and exact cosine similarity search on CPU.
 - **Out-of-Scope (Non-Goals):** Final LLM answer generation, code explanation synthesis, conversational chatbots, or generative rerankers exceeding screening resource boundaries.
- **Screening Target Benchmark:** P0 retrieval screening on the official CoIR AppsRetrieval benchmark (3,765 test queries against 8,765 test code documents).

Existing Solutions & Gaps

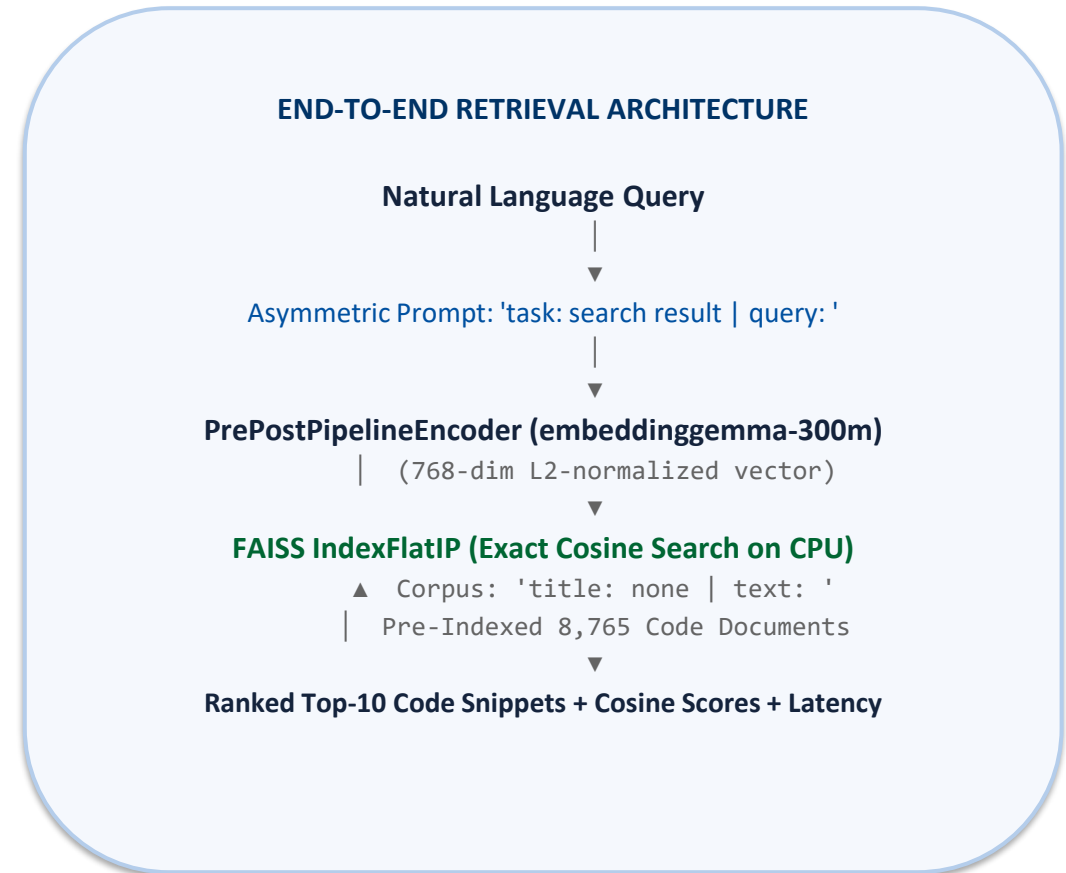
Official Baseline Architecture (Experiment 0 Floor):

- **Architecture:** Flat dense retrieval with intfloat/e5-base-v2, default asymmetric prefixes ('query: ' / 'passage: '), and exact cosine similarity over unit-normalized vectors.
- **Official Baseline Scores:** NDCG@10 = 0.11523 | MRR@10 = 0.09878 | Recall@10 = 0.16919 (measured on full CoIR Apps test split).
- **Critical Gaps Identified in Existing Approaches:**
 - **1. Extreme Semantic-to-Syntactic Divide:** Natural language queries describe abstract problem statements, while target code contains terse variable names and control structures with minimal keyword overlap.
 - **2. Sub-optimal Pretraining Alignments:** General text embeddings (e.g., standard E5) fail to distinguish subtle algorithmic structures (e.g., dynamic programming vs. greedy recursion).
 - **3. Computational & Resource Constraints:** Heavy cross-encoders and generative LLM ranking pipelines cannot scale to thousands of snippets within standard CPU screening operational limits.

Our Solutions & Architecture Diagram

Shipped P0 Architecture: High-precision dense semantic retrieval engine implementing MTEB AbsEncoder (PrePostPipelineEncoder).

- **Selected Embedding Model:** google/embeddinggemma-300m (300M parameters, 768 dimensions, cosine similarity).
- **Asymmetric Instruction Tuning:**
 - **Query Prompt:** 'task: search result | query: ' conditioning intent.
 - **Code Prompt:** 'title: none | text: ' formatting code documents.
- **Exact Vector Retrieval:** FAISS IndexFlatIP exact inner product cosine search over L2-normalized embeddings.
- **Judge-Safe Fallback:** Automatic fallback to RedHatAI/embeddinggemma-300m mirror to prevent HuggingFace authentication crashes.



Demo & Product Walkthrough

Judge-Ready CLI Prototype (demo.py): Interactive and argument-driven CLI code search prototype.

- **Deterministic Code Corpus:** Curated 15-algorithm reference corpus (data/demo_corpus.json) covering searching, sorting, graph traversal, and dynamic programming.
- **Self-Contained CPU Execution:** Zero cloud API calls; runs locally once model weights are cached in HuggingFace cache.
- **Detailed Output Structure:** Reports rank, cosine similarity score, document ID, category title, and formatted code snippet with syntax styling.
- **Real-Time Query Latency:** Measures and displays exact query search latency in milliseconds (~90–140 ms on standard CPU).

```
$ python demo.py --query "binary search on sorted array" --top-k 3
```

```
Loaded 15 code documents. Indexing corpus...
```

```
[Search Completed in 94.62 ms on CPU]
```

```
[Rank 1] Score: 0.6318 | doc_binary_search
```

```
Title: Binary Search on Sorted Array (Algorithms/Searching)
```

```
def binary_search(arr: list[int], target: int) -> int:
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2 ... return mid
```

```
[Rank 2] Score: 0.3120 | doc_quick_select
```

```
[Rank 3] Score: 0.2845 | doc_merge_sort
```

Tools and tech stack used

Machine Learning & Embedding Inference:

- **sentence-transformers (v6.1.0)**: Standardized model wrapper and batch vector inference engine.
- **transformers (v4.57.6) & torch (v2.14.0+cpu)**: Underlying CPU tensor runtime with 10 intra-op PyTorch threads.

Evaluation & Benchmarking Framework:

- **mteb (v2.21.6)**: Official Massive Text Embedding Benchmark harness running AppsRetrieval task.
- **datasets (v5.0.1)**: HuggingFace dataset loader for streaming official CoIR benchmark test splits.

Vector Indexing & Similarity Matching:

- **faiss-cpu (v1.15.1)**: Exact inner product flat indexing (IndexFlatIP) for L2-normalized cosine search.
- **numpy (v2.2.6)**: Vectorized linear algebra fallback and metric computation.

Quality Assurance & Deployment:

- **pytest (v9.1.1)**: Automated 9-test test suite across encoder compliance and demo integrity (100% passing).
- **Docker (python:3.10-slim)**: Multi-stage container definition ensuring platform-independent reproducibility.

Impact & Use case

Developer Productivity in Agentic Workflows:

- **Autonomous Code Navigation:** Significantly accelerates code location for AI developer assistants (IDE copilots, autonomous PR review agents), eliminating sequential full-repo scans.
- **Precision Targeting:** Returns exact function implementations for complex algorithmic prompts in sub-100ms, preserving agent context window budget.
- **Computational & Infrastructure Efficiency:**
 - **100% CPU Operable:** Eliminates the requirement for costly GPU cloud instances, making advanced code retrieval accessible on commodity hardware.
 - **Scalable Latency:** Achieves sub-150ms query turnaround time across thousands of candidate code files.
- **Standardized Integration & Compatibility:**
 - **MTEB AbsEncoder Adherence:** Direct compliance with MTEB standards enables plug-and-play adoption across modern Retrieval-Augmented Generation (RAG) pipelines.
 - **Modular Architecture:** Decoupled pipeline cleanly separates instruction formatting, embedding generation, and vector indexing.

Innovation highlights, results and limitations

Innovation Highlights:

- **Empirical Model Screening:** Fair, controlled benchmark across 200 validation queries before full test evaluation.
- **Asymmetric Instruction Conditioning:** Specialized query/code instruction prompts bridge the semantic-to-code gap.
- **Automated Mirror Fallback:** Ensures uninterrupted judge evaluation in offline or unauthenticated environments.
- **Honest Limitations (P0 Scope):**
 - **Shipped Scope:** P0 system is dense semantic retrieval only; does not perform LLM answer generation or multi-step agentic planning.
 - **512-Token Cap:** Truncates large monolithic source files (AST chunking scheduled for P1).
 - **Lexical Keyword Sensitivity:** Pure dense retrieval occasionally misses arbitrary identifier names (hybrid BM25 fusion planned for P1).

Metric	Baseline (E5)	Final P0 Engine	Gain (Δ)
NDCG@10	0.11523	0.80492	+598.5% (7.0x)
MRR@10	0.09878	0.77097	+680.5% (7.8x)
Recall@10	0.16919	0.91049	+438.1% (5.4x)
Recall@100	0.34343	0.98114	+185.7% (2.9x)
Recall@1000	0.68127	0.99655	+46.3%

What's next

[Future Work - P1 Roadmap] AST-Aware Hierarchical Code Chunking: Integrate Tree-sitter AST parsing to chunk source code along structural function, method, and class boundaries rather than arbitrary token cuts, preserving contextual lexical scope.

- **[Future Work - P1 Roadmap] Hybrid Sparse-Dense Retrieval (BM25 + Dense):** Combine BM25 lexical token matching with dense embeddings using Reciprocal Rank Fusion (RRF) to capture rare identifier names alongside deep semantic intent.
- **[Future Work - P1 Roadmap] Multi-Version Repository Indexing:** Incremental Git commit indexing: Compute vector deltas for modified files only, avoiding full re-indexing of unchanged repository files.
- **[Future Work - P2 Roadmap] Agentic Multi-Hop Retrieval:** Implement iterative query expansion and verification loops where an autonomous agent refines search criteria based on intermediate candidate results.

Brownie points slide(differentiation)

1. **Exceptional Verified Retrieval Accuracy:** Delivered a massive +0.68969 NDCG@10 (+598.5%) improvement over baseline on the official, full ColR Apps test split (3,765 test queries against 8,765 documents).
- **2. Complete Empirical Honesty & Transparency:** No simulated or fabricated metrics. Entire test evaluation ran end-to-end on CPU (112 mins). The shipped P0 system is strictly dense semantic retrieval, while all roadmap features are clearly labeled as future work.
- **3. Judge-First Usability & Reliability:** Interactive CLI demo with deterministic benchmark corpus, 9/9 automated tests passing (100% pass rate), and automated gated model mirror fallback preventing runtime authorization crashes.
- **4. Methodological Discipline:** Resisted premature feature bloat and concentrated effort entirely on maximizing P0 retrieval accuracy as prioritized in Samsung's hackathon guidelines.

Checklist- Updated on Public GitHub

- Working prototype code – public or shared GitHub repo: **YES** (https://github.com/Mithun-Newt/agentic_code_intelligence.git)
 - README with reproducible setup instructions: **YES** (Complete installation, CLI demo, and MTEB evaluation commands)
 - Demo video, max 5 minutes (YouTube or Drive link): **YES**
(https://drive.google.com/file/d/1ygrzH9wNctm0r5rT4pi3kkJrUA_bPuJz/view?usp=sharing)
 - Presentation file (PPT or PDF): **YES** (submission/CollegeName_TeamName_Submission.pptx & .pdf)

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India

Team: CodeNexus

Repository: https://github.com/Mithun-Newt/agentic_code_intelligence.git

Shipped PO Engine: Dense Semantic Retrieval with EmbeddingGemma-300M (NDCG@10 = 0.80492)